Name: Sadia sohail

Student id: CSC-25F-634

Subject: object - oriented programming

Course instructor: sir Majid kaleem

Assignment : 01

Question : 01

a) **Define the four pillars of OOP (Abstraction, Encapsulation, Inheritance, Polymorphism) in your own words. Provide one real-world analogy for each.**

1. Abstraction:

Abstraction mean showing only the important feature of an object and hiding the complex detail behind it.it help us to focus on what an object does it instead of how it works internally.

Example:

Like using a remote control:

You can change the channels or volume without knowing how the internal circuits works .

2. Enacapsulation:

Encapsulation means combining data into a single class and protected the data from direct access .It makes sure that data is only changed through methods.

Example:

Like a closed box:

Where the contents are hidden and cannot be accessed directly.

3. Inheritance:

Inheritance means one class use use the properties and behaviour of another class. It helps reusing the code and create a relationship between class and object.

Example:

Like a child:

Inheriting qualities from their parents such as eye colour , skin tone etc.

4. Polymorphism:

Polymorphism means one method can behave different way depending on the situation.it allows the same action to perform differently.

Example:

Like a single person:

1. · A person can be a teacher in school.
2. · A parent at home .

3. · A friend among friends.

b) Explain the role of access modifiers (public, private, protected, and default) in Java. How do they support encapsulation?

Access modifiers in java :

Access modifiers are key words in java that controls who can access variable, methods and classes.They help keep data safe and organized.

How do they support encapsulation?

· Make variables private so other classes cannot change them directly.
· This protects the data from being used in the wrong way.
· Use public methods (getter and setter) to access or update the data.
· These methods control how the data is used.
· This keeps the data safe and used properly.

c) Compare Encapsulation vs. Abstraction with examples.

compression of encapsulation and abstraction.	
encapsulation	**abstraction**
● hiding data and protect it.	● hide complex details and showing only important features.
● using private variables and getter / setter methods	● using abstract classes.
● private variables marks with public method to access it.	● shape class with abstract draw() method, implemented by circle or square.

●
class Student {
private int marks; //hidden
public int getMarks() { return marks; }
public void setMarks(int m) { marks = m; }
}
 	
public class Main {
public static void main(String[] args) {
Student s = new Student();
s.setMarks(95);
System.out.println(s.getMarks());
}
	●
	abstract class Shape {
	abstract void draw(); // abstract method, no body
	}
	class Circle extends Shape {
	void draw() {
	System.out.println("Drawing Circle");
	}
	}
	public class Main {
	public static void main(String[] args) {
	Shape s = new Circle(); // using abstraction

}	s.draw();
	}
	}

Question : 2

You are designing a simple University Management System. The system should manage Students and Courses.

a) Create a base class Person with attributes name and age.

```
class Person {
   String name;
   int age;
   Person(String name, int age) {
      this.name = name;
      this.age = age;
   }
   void display() {
      System.out.println("Name: " + name);
      System.out.println("Age: " + age);
   }
}
public class Main {
   public static void main(String[] args) {
      Person p = new Person("Alice", 25);
      p.display();
   }
}
```

b) Use appropriate access modifiers for attributes.

```
class Person {
   private String name;
   private int age;
     public Person(String name, int age) {
      this.name = name;
      this.age = age;
   }
    public String getName() {
      return name;
   }
     public void setName(String name) {
      this.name = name;
   }
   public int getAge() {
      return age;
   }
   public void setAge(int age) {
      this.age = age;
```

```
    }
    public void display() {
        System.out.println("Name: " + name);
        System.out.println("Age: " + age);
    }
}
public class Main {
    public static void main(String[] args) {
        Person p = new Person("sadia", 18);
             System.out.println("Name: " + p.getName());
                 System.out.println("Age: " + p.getAge());
            p.setName("sohail");
            p.setAge(20);
            p.display();
    }
}
```

c) Provide getter and setter methods.

```
class Person {
    private String name;
    private int age;
      public Person(String name, int age) {
        this.name = name;
        this.age = age;
    }

    // Public getter and setter for name
    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    // Public getter and setter for age
    public int getAge() {
        return age;
    }

    public void setAge(int age) {
        this.age = age;
    }

    public void display() {
        System.out.println("Name: " + name);
        System.out.println("Age: " + age);
    }
```

```
}

public class Main {
    public static void main(String[] args) {
        Person p = new Person("sadia", 18);

        // Accessing attributes using getters
        System.out.println("Name: " + p.getName());
        System.out.println("Age: " + p.getAge());

        // Updating attributes using setters
        p.setName("sadia");
        p.setAge(20);

        p.display();
    }
}
```

(d) Create a derived class Student that inherits from Person.


```
class Person {
    private String name;
    private int age;

    public Person(String name, int age) {
        this.name = name;
        this.age = age;
    }

    public String getName() { return name; }
    public int getAge() { return age; }
}

class Student extends Person {
    private String course;

    public Student(String name, int age, String course) {
        super(name, age);
        this.course = course;
    }

    public String getCourse() { return course; }
}

public class Main {
    public static void main(String[] args) {
        Student s = new Student("sadia", 19, "Computer Science");
```

```
        System.out.println("Name: " + s.getName());
        System.out.println("Age: " + s.getAge());
        System.out.println("Course: " + s.getCourse());
    }
}
```

e) Add attributes: studentId and department.

```
public class Student {

    private int studentId;
    private String department;

    public static void main(String[] args) {
        Student s = new Student();
        s.studentId = 634;
        s.department = "Computer Science";

        System.out.println("ID: " + s.studentId);
        System.out.println("Department: " + s.department);
    }
}
```

f) Create a class Course with attributes course Code, courseName, and creditHours.

```
class Course {
    String courseCode;
    String courseName;
    int creditHours;


    Course(String code, String name, int hours) {
        courseCode = code;
        courseName = name;
        creditHours = hours;
    }
}

public class Main {
    public static void main(String[] args) {
        Course c1 = new Course("CSC104", "oop", 3);

        System.out.println(c1.courseCode);
        System.out.println(c1.courseName);
        System.out.println(c1.creditHours);
    }
```

}

g) Apply encapsulation using private attributes and public methods

```
// Encapsulated Course class
class Course {
private String courseCode; // private attribute
private String courseName; // private attribute
private int creditHours; // private attribute

// Public setter methods
public void setCourseCode(String code) {
courseCode = code;
}

public void setCourseName(String name) {
courseName = name;
}

public void setCreditHours(int hours) {
creditHours = hours;
}

// Public getter methods
public String getCourseCode() {
return courseCode;
}

public String getCourseName() {
return courseName;
}

public int getCreditHours() {
return creditHours;
}
}

// Main class to test encapsulation
public class Main {
public static void main(String[] args) {
Course c1 = new Course();

// Set values using setters
c1.setCourseCode("CS101");
c1.setCourseName("Programming");
c1.setCreditHours(3);
```

```
// Access values using getters
System.out.println("Course Code: " + c1.getCourseCode());
System.out.println("Course Name: " + c1.getCourseName());
System.out.println("Credit Hours: " + c1.getCreditHours());
}
}
```

h) Demonstrate abstraction by creating an abstract class Evaluation with an abstract method calculateGrade()

```
abstract class Evaluation {
public abstract char calculateGrade(int marks);
}

// Subclass that implements the abstract method
class Exam extends Evaluation {
@Override
public char calculateGrade(int marks) {
if (marks >= 90) return 'A';
else if (marks >= 80) return 'B';
else if (marks >= 70) return 'C';
else if (marks >= 60) return 'D';
else return 'F';
}
}
public class Main {
public static void main(String[] args) {
Exam exam1 = new Exam();

int marks = 85;
char grade = exam1.calculateGrade(marks);

System.out.println("Marks: " + marks);
System.out.println("Grade: " + grade);
}
}
```

i) Implement this in a subclass ExamEvaluation.

```
abstract class Evaluation {
// Abstract method
public abstract char calculateGrade(int marks);
}

// Subclass that implements the abstract method
class ExamEvaluation extends Evaluation {
@Override
public char calculateGrade(int marks) {
if (marks >= 90) return 'A';
else if (marks >= 80) return 'B';
```

```
else if (marks >= 70) return 'C';
else if (marks >= 60) return 'D';
else return 'F';
}
}

public class Main {
public static void main(String[] args) {
ExamEvaluation exam = new ExamEvaluation();

int marks1 = 92;
int marks2 = 75;

System.out.println("Marks: " + marks1 + " -> Grade: " +
exam.calculateGrade(marks1));
System.out.println("Marks: " + marks2 + " -> Grade: " +
exam.calculateGrade(marks2));
}
}
```